ALAB 351.4 - Functions, Tuples, Dictionaries, and Exceptions

Name: James Sloan
Course: Python Essentials (UCI 3052)
Date: August 27, 2026

Repository

All files for this lab are in a single GitHub repository:

https://github.com/santigrey/Python-Essentials

The scripts for this lab are in `modules/Module 351/ALAB-351.4/`:

https://github.com/santigrey/Python-Essentials/tree/main/modules/Module%20351/ALAB-351.4

Files

File	Part
`basic_functions.py`	Part 1 — Writing and Using Functions
`calc_with_functions.py`	Part 1 — Calculator Refactored With Functions
`tuples_dicts.py`	Part 2 — Tuples and Dictionaries
`data_processing.py`	Part 2 — Data Processing
`exception_demo.py`	Part 3 — Exception Handling

basic_functions.py

Part 1 — Writing and Using Functions

Code

```
# ALAB 351.4 - Part 1: Writing and Using Functions
# James Sloan
#
# Defines three small functions and demonstrates each one.


# ------------------------------------------------------------------------
# greet_user() prints a greeting for the name it is given.
#
# name is a parameter with a default value of "". Giving it a default is what
# lets me call greet_user() with no argument at all - Python fills in the ""
# for me. An empty string is falsy, so "if name" is False when nothing was
# passed and the shorter greeting runs instead.
#
# This function prints; it has no return statement, so it hands back None.
# ------------------------------------------------------------------------
def greet_user(name=""):
    if name:
        print(f"Hello, {name}! Welcome!")
    else:
        print("Hello! Welcome!")


# ------------------------------------------------------------------------
# add_two_numbers() takes two numbers and returns their sum.
#
# Parameters: a and b, the two numbers to add.
# Returns: the sum. "return" hands the value back to whoever called the
# function, which means the answer can be stored in a variable or used inside
# a bigger expression.
# ------------------------------------------------------------------------
def add_two_numbers(a, b):
    return a + b


# ------------------------------------------------------------------------
# is_even() reports whether a number is even.
#
# Parameter: num, the number to test.
# Returns: True if num is even, False if it is not.
# num % 2 is the remainder after dividing by 2. That remainder is 0 for even
```

```
# numbers, so "num % 2 == 0" is already True or False - I can return the
# comparison directly instead of writing an if/else that returns True or False.
# ------------------------------------------------------------------------------
def is_even(num):
    return num % 2 == 0



# ------------------------------------------------------------------------------
# Main part of the script - everything below here is outside the functions.
# ------------------------------------------------------------------------------
print("Part 1: greet_user()")
print("--------------------")
greet_user("James")          # with a name
greet_user()                 # without a name, so the default "" is used
print()

print("Part 2: add_two_numbers()")
print("-------------------------")
# Storing the returned value in a variable.
total = add_two_numbers(12, 30)
print(f"add_two_numbers(12, 30) returned {total}")

# Using the returned value inside a larger expression.
print(f"Doubling that result gives {total * 2}")
print(f"Adding two returned values: {add_two_numbers(1, 2) + add_two_numbers(3, 4)}")
print()

print("Part 3: is_even()")
print("-----------------")
even_check = is_even(4)
odd_check = is_even(7)
print(f"4 is even: {even_check}")
print(f"7 is even: {odd_check}")

# The returned value is a real boolean, so it can drive an if statement.
if is_even(10):
    print("10 is even, so this line runs.")
```

Output

```
Part 1: greet_user()
--------------------
Hello, James! Welcome!
Hello! Welcome!

Part 2: add_two_numbers()
-------------------------
add_two_numbers(12, 30) returned 42
Doubling that result gives 84
Adding two returned values: 10

Part 3: is_even()
-----------------
4 is even: True
7 is even: False
10 is even, so this line runs.
```

---

calc_with_functions.py

Part 1 — Calculator Refactored With Functions

Code

```
# ALAB 351.4 - Part 1: Calculator Refactored With Functions
# James Sloan
#
# The Module 2 calculator rebuilt so each operation is its own function.
# calculate() picks the right one, and try/except handles bad input and
# division by zero.


# ------------------------------------------------------------------------
# One function per operation. Each takes two numbers and returns the result.
#
# divide() does not check for a zero divisor itself. Dividing by zero raises
# ZeroDivisionError, and letting that error travel back to the caller is the
# point of Part 1 - the main program catches it instead.
# ------------------------------------------------------------------------
def add(a, b):
    return a + b


def subtract(a, b):
    return a - b


def multiply(a, b):
    return a * b


def divide(a, b):
    return a / b


# ------------------------------------------------------------------------
# calculate() is a function that calls other functions.
#
# Parameters: a and b are the numbers, op is the operation symbol.
# Returns: the result of the matching operation, or an error message string
# if op is not one of the four symbols.
#
# The if/elif chain matches the symbol and hands the work to the right
# function above. Only one branch can run, and each one returns immediately.
# ------------------------------------------------------------------------
```

```
def calculate(a, b, op):
    if op == "+":
        return add(a, b)
    elif op == "-":
        return subtract(a, b)
    elif op == "*":
        return multiply(a, b)
    elif op == "/":
        return divide(a, b)
    else:
        return f"Error: '{op}' is not a valid operation."


# ------------------------------------------------------------------------
# tidy() keeps the printing readable.
# float() turns every input into a float, so 7 comes back as 7.0. If the
# number is really whole, print the int version so the line reads 21, not 21.0.
# ------------------------------------------------------------------------
def tidy(number):
    return int(number) if number == int(number) else round(number, 4)


# ------------------------------------------------------------------------
# Main program.
# ------------------------------------------------------------------------
print("Calculator With Functions")
print("=========================")

# try/except around the input conversion. float() raises ValueError when the
# text is not a number, so both numbers are read inside the same try block.
try:
    first = float(input("Enter the first number: "))
    second = float(input("Enter the second number: "))
except ValueError:
    print("Error: that is not a valid number.")
else:
    operation = input("Choose an operation (+, -, *, /): ").strip()

    # try/except around the calculation itself, which is where dividing by
    # zero fails.
    try:
        result = calculate(first, second, operation)
    except ZeroDivisionError:
```

```
        print("Error: division by zero is not allowed.")
    else:
        # An invalid symbol makes calculate() return a message instead of a
        # number, so check the type before formatting it as an equation.
        if isinstance(result, str):
            print(result)
        else:
            print(f"{tidy(first)} {operation} {tidy(second)} = {tidy(result)}")
```

Output

A valid calculation:

```
Calculator With Functions
=========================
Enter the first number: 12
Enter the second number: 4
Choose an operation (+, −, ∗, /): ∗
12 ∗ 4 = 48
```

Division by zero, caught by `except ZeroDivisionError`:

```
Calculator With Functions
=========================
Enter the first number: 9
Enter the second number: 0
Choose an operation (+, −, ∗, /): /
Error: division by zero is not allowed.
```

Invalid numeric input, caught by `except ValueError`:

```
Calculator With Functions
=========================
Enter the first number: abc
Error: that is not a valid number.
```

An operation symbol that is not one of the four:

```
Calculator With Functions
=========================
Enter the first number: 8
Enter the second number: 2
Choose an operation (+, -, *, /): %
Error: '%' is not a valid operation.
```

---

tuples_dicts.py

Part 2 — Tuples and Dictionaries

Code

```
# ALAB 351.4 - Part 2: Tuples and Dictionaries
# James Sloan
#
# Shows that a tuple cannot be changed, then builds a dictionary and adds to,
# updates, and loops over it.


# ------------------------------------------------------------------------------
# A tuple is written with parentheses. Once it exists it cannot be changed.
# ------------------------------------------------------------------------------
months = ("January", "February", "March", "April", "May", "June",
          "July", "August", "September", "October", "November", "December")

print("Tuples")
print("======")
print(f"There are {len(months)} months in the tuple.")

# Index 0 is the first item. Index -1 counts back from the end, so it is the
# last item without needing to know the length.
print(f"First month: {months[0]}")
print(f"Last month:  {months[-1]}")
print()

# ------------------------------------------------------------------------------
# Trying to change a tuple raises TypeError. Wrapping it in try/except lets
# the script show the error and keep running instead of crashing.
# ------------------------------------------------------------------------------
try:
    months[0] = "NewMonth"
except TypeError as error:
    print(f"Tuples are immutable, error: {error}")
print()


# ------------------------------------------------------------------------------
# A dictionary stores key/value pairs. Here the key is a student name and the
# value is that student's grade.
# ------------------------------------------------------------------------------
students = {
    "Alice": 90,
    "Brian": 82,
```

```
    "Chloe": 95,
    "Diego": 78,
}

print("Dictionaries")
print("============")
print(f"Starting dictionary: {students}")
print()

# Assigning to a key that does not exist yet adds a new pair.
students["Elena"] = 88
print("After adding Elena:")
print(students)
print()

# Assigning to a key that already exists replaces its value instead.
students["Brian"] = 91
print("After updating Brian's grade:")
print(f"Brian: {students['Brian']}")
print()

# ------------------------------------------------------------------------
# .items() hands back both the key and the value on each pass, so the loop
# variables name and grade are filled in together.
# ------------------------------------------------------------------------
print("All students:")
for name, grade in students.items():
    print(f"{name}: {grade}")
```

Output

```
Tuples
======
There are 12 months in the tuple.
First month: January
Last month:  December

Tuples are immutable, error: 'tuple' object does not support item assignment

Dictionaries
============
Starting dictionary: {'Alice': 90, 'Brian': 82, 'Chloe': 95, 'Diego': 78}

After adding Elena:
{'Alice': 90, 'Brian': 82, 'Chloe': 95, 'Diego': 78, 'Elena': 88}

After updating Brian's grade:
Brian: 91

All students:
Alice: 90
Brian: 91
Chloe: 95
Diego: 78
Elena: 88
```

---

data_processing.py

Part 2 — Data Processing

Code

```
# ALAB 351.4 - Part 2: Data Processing
# James Sloan
#
# Averages the grades stored in a dictionary of courses. One course has no
# grades at all, which is the edge case the exception handling is there for.


# ------------------------------------------------------------------------
# get_average_grade() averages a tuple of numbers.
#
# Parameter: grades_tuple, a tuple of numeric grades.
# Returns: the average, or None if the tuple is empty.
#
# sum() adds the grades and len() counts them. If the tuple is empty len() is
# 0, and dividing by 0 raises ZeroDivisionError. Catching it and returning
# None means the caller gets an answer it can check rather than a crash.
# ------------------------------------------------------------------------
def get_average_grade(grades_tuple):
    try:
        return sum(grades_tuple) / len(grades_tuple)
    except ZeroDivisionError:
        print("  Warning: no grades to average.")
        return None


# ------------------------------------------------------------------------
# The keys are course names and the values are tuples of grades.
# Art is deliberately empty to trigger the edge case above.
# ------------------------------------------------------------------------
course_grades = {
    "Math": (88, 92, 79, 85, 90),
    "Science": (75, 80, 68, 91),
    "History": (95, 89, 100),
    "Art": (),
}

print("Course Averages")
print("===============")

for course, grades in course_grades.items():
    average = get_average_grade(grades)
```

```
    # get_average_grade() returns None for the empty course, so check for that
    # before trying to format the number.
    if average is None:
        print(f"The average grade for {course} could not be calculated.")
    else:
        # :.1f rounds to one decimal place for display.
        print(f"The average grade for {course} is {average:.1f}")
```

Output

```
Course Averages
===============
The average grade for Math is 86.8
The average grade for Science is 78.5
The average grade for History is 94.7
  Warning: no grades to average.
The average grade for Art could not be calculated.
```

---

exception_demo.py

Part 3 — Exception Handling

Code

```
# ALAB 351.4 - Part 3: Exception Handling
# James Sloan
#
# Raises an exception on purpose, catches it, and shows that a finally clause
# runs either way. Also catches a generic Exception.


# ------------------------------------------------------------------------
# safe_divide() divides a by b.
#
# Parameters: a is the numerator, b is the divisor.
# Returns: a / b when b is not zero.
# Raises: ValueError when b is zero.
#
# "raise" creates the error myself instead of waiting for Python to do it.
# The message in the parentheses is what the except block can print later.
# ------------------------------------------------------------------------
def safe_divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b


print("Raising and catching ValueError")
print("==============================")

# Two test pairs - the second one has a zero divisor, so it fails.
test_values = [(10, 2), (7, 0)]

for a, b in test_values:
    print(f"Trying {a} / {b}")
    try:
        answer = safe_divide(a, b)
        print(f"  Result: {answer}")
    except ValueError as error:
        # "as error" gives me the exception object so I can print its message.
        print(f"  Error: {error}")
    finally:
        # finally always runs - after a success and after a caught error.
        print("  Division operation completed")
    print()
```

```
# ------------------------------------------------------------------------
# Catching a generic Exception.
#
# int() raises ValueError on text that is not a number. Catching Exception
# instead of ValueError works because ValueError is a kind of Exception, so a
# generic catch picks up anything the block might throw.
# ------------------------------------------------------------------------
print("Catching a generic Exception")
print("============================")

try:
    number = int("not a number")
    print(f"Converted value: {number}")
except Exception as error:
    print(f"Something went wrong: {error}")
    print(f"The exception type was: {type(error).__name__}")
```

Output

```
Raising and catching ValueError
===============================
Trying 10 / 2
  Result: 5.0
  Division operation completed

Trying 7 / 0
  Error: Cannot divide by zero
  Division operation completed

Catching a generic Exception
============================
Something went wrong: invalid literal for int() with base 10: 'not a number'
The exception type was: ValueError
```

Notes on what I learned

Part 1 — Functions

Giving `greet_user()` a default parameter value is what lets the same function handle both calls. `def greet_user(name="")` means calling it with no argument fills in the empty string, and an empty string is falsy, so `if name` picks the shorter greeting on its own.

`return` is the difference between the three functions. `greet_user()` prints and returns nothing, so it hands back `None`. `add_two_numbers()` and `is_even()` return values, which is why their results can be stored in a variable (`total = add_two_numbers(12, 30)`) or used inside a bigger expression (`add_two_numbers(1, 2) + add_two_numbers(3, 4)`).

`is_even()` returns the comparison `num % 2 == 0` directly. That comparison is already `True` or `False`, so writing `if ...: return True else: return False` around it would be doing the same work twice.

Refactoring the Module 2 calculator moved the four operations out of one long `if/elif` chain and into four named functions. `calculate()` then calls whichever one matches — a function calling other functions. The logic is the same as Module 2, but each piece can now be read on its own.

Part 2 — Tuples and Dictionaries

Trying to assign into a tuple raises `TypeError: 'tuple' object does not support item assignment`. Catching it proves immutability without stopping the script. Index `-1` was the useful part of reading the tuple — it gets the last month without needing to know there are twelve.

Dictionaries use the same square-bracket syntax for two different jobs. `students["Elena"] = 88` adds a new pair because that key did not exist, and `students["Brian"] = 91` replaces a value because that key did. Nothing in the syntax tells you which is happening; only whether the key already exists does.

`.items()` gives back the key and the value together, which is what makes `for name, grade in students.items()` work.

Part 3 — How exceptions were caught and handled

Each script catches a different failure, and the type matters:

Script	Exception	How it was handled
`calc_with_functions.py`	`ValueError`	`float()` on text that is not a number
`calc_with_functions.py`	`ZeroDivisionError`	raised by `divide()`, caught in the main program
`tuples_dicts.py`	`TypeError`	raised by assigning into a tuple
`data_processing.py`	`ZeroDivisionError`	empty grade tuple, returns `None` instead
`exception_demo.py`	`ValueError`	raised on purpose with `raise`
`exception_demo.py`	`Exception`	generic catch around `int("not a number")`

The part that clicked was that I have two choices about where to handle an error. `divide()` does not check for a zero divisor itself — it lets `ZeroDivisionError` travel back to the caller, which catches it. `get_average_grade()` does the opposite: it catches its own `ZeroDivisionError` and returns `None`, so the caller checks for `None` rather than handling an exception. Both work; the second one turns a crash into a value the rest of the program can test.

`finally` runs either way. In `exception_demo.py` the same "Division operation completed" line prints after the successful `10 / 2` and after the failed `7 / 0`.

Catching the generic `Exception` works on the `int("not a number")` line because `ValueError` is a kind of `Exception`, so the broad catch picks it up. Printing `type(error).__name__` confirmed it really was a `ValueError` underneath. A broad catch is convenient but it hides which error actually happened, so naming the specific exception is the better habit where I know what can go wrong.